Watching for users

Sign up as your first user

Install the SDK, run your dev server, and sign yourself up. The moment we detect that first account, installation is complete.

Next.js | React | TanStack Start | Expo | iOS | JavaScript | Astro

> **Important**
> Ensure that the Native API is enabled to integrate Clerk in your native application. In the Clerk Dashboard, navigate to the Native applications page.

1 Create a new Expo app

Run the following commands to create a new Expo app:

npm pnpm yarn bun

terminal

```
npx create-expo-app@latest clerk-expo
cd clerk-expo
```

2 Remove default template files

The default Expo template includes files that will conflict with the routes you'll create in this guide. Remove the conflicting files and unused `components/` directory:

npm pnpm yarn bun

terminal

```
rm -rf "app/(tabs)" app/modal.tsx app/+not-found.tsx components/
```

The default template also includes `react-native-reanimated`, which can cause known Android build issues. Since it's not needed for this guide, remove it to avoid build errors:

npm pnpm yarn bun

terminal

```
npm uninstall react-native-reanimated react-native-worklets --legacy-peer-deps
```

Then, remove the `import 'react-native-reanimated'` line from `app/_layout.tsx`.

> **Important**

> You can skip this step if you used `npx create-expo-app@latest --template blank` to create your app. However, the blank template doesn't include Expo Router or pre-styled UI components. You'll need to install `expo-router` and its dependencies to follow along with this guide.

3 Install dependencies

Install the required packages. Use `npx expo install` to ensure SDK-compatible versions.

- The Clerk Expo SDK gives you access to prebuilt components, hooks, and helpers to make user authentication easier.
- Clerk stores the active user's session token in memory by default. In Expo apps, the recommended way to store sensitive data, such as tokens, is by using `expo-secure-store` which encrypts the data before storing it.

npm pnpm yarn bun

terminal

```
npm install @clerk/expo expo-secure-store
```

4 Set your Clerk API keys

Add your Clerk Publishable Key to your `.env` file or create the file if it doesn't exist. Retrieve these keys anytime from the API keys page.

.env

```
EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY=pk_test_c2VOdGxlZC1tb29zZSOyNC5jbGVyay5hY2NvdW5Ocy5kZXYk
```

5 Add `<ClerkProvider>` to your root layout

The `ClerkProvider` component provides session and user context to Clerk's hooks and components. It's recommended to wrap your entire app at the entry point with `ClerkProvider` to make authentication globally accessible. See the reference docs for other configuration options.

Add the component to your root layout and pass your Publishable Key and `tokenCache` as props, as shown in the following example:

app/_layout.tsx

```
import { ClerkProvider } from '@clerk/expo'
import { tokenCache } from '@clerk/expo/token-cache'
import { Slot } from 'expo-router'

const publishableKey = process.env.EXPO_PUBLIC_CLERK_PUBLISHABLE_KEY!

if (!publishableKey) {
  throw new Error('Add your Clerk Publishable Key to the .env file')
}

export default function RootLayout() {
  return (
    <ClerkProvider publishableKey={publishableKey} tokenCache={tokenCache}>
      <Slot />
    </ClerkProvider>
  )
}
```

6 Protect your `auth` routes in the `layout`

Clerk currently only supports control components for Expo native. UI components are only available for Expo web. Instead, you must build custom flows using Clerk's API. The following sections demonstrate how to build custom email/password sign-up and sign-in flows. If you want to use different authentication methods, such as passwordless or OAuth, see the dedicated custom flow guides.

First, protect your sign-up and sign-in pages. Create a new route group `(auth)` with a `_layout.tsx` file with the following code. The `useAuth()` hook is used to access the user's authentication state. If the user is already signed in, they will be redirected to the home page.

app/(auth)/_layout.tsx

```
import { useAuth } from '@clerk/expo'
import { Redirect, Stack } from 'expo-router'

export default function AuthRoutesLayout() {
  const { isSignedIn, isLoaded } = useAuth()

  if (!isLoaded) {
    return null
  }

  if (isSignedIn) {
    return <Redirect href={'/'} />
  }

  return <Stack />
}
```

7 Add `sign-up` page

In the `(auth)` route group, create a `sign-up.tsx` file with the following code. The `useSignUp()` hook is used to create a sign-up flow. The user can sign up using their email and password and will receive an email verification code to confirm their email.

app/(auth)/sign-up.tsx

```
import { ThemedText } from '@/components/themed-text'
import { ThemedView } from '@/components/themed-view'
import { useAuth, useSignUp } from '@clerk/expo'
import { type Href, Link, useRouter } from 'expo-router'
import React from 'react'
import { Pressable, StyleSheet, TextInput, View } from 'react-native'

export default function Page() {
  const { signUp, errors, fetchStatus } = useSignUp()
  const { isSignedIn } = useAuth()
  const router = useRouter()

  const [emailAddress, setEmailAddress] = React.useState('')
  const [password, setPassword] = React.useState('')
  const [code, setCode] = React.useState('')

  const handleSubmit = async () => {
    const { error } = await signUp.password({
      emailAddress,
      password,
    })
    if (error) {
      console.error(JSON.stringify(error, null, 2))
      return
    }

    if (!error) await signUp.verifications.sendEmailCode()
  }

  const handleVerify = async () => {
    await signUp.verifications.verifyEmailCode({
      code,
    })
    if (signUp.status === 'complete') {
      await signUp.finalize({
```

```
      // Redirect the user to the home page after signing up
      navigate: ({ session, decorateUrl }) => {
        if (session?.currentTask) {
          // Handle pending session tasks
          // See https://clerk.com/docs/guides/development/custom-flows/authentication/session-tasks
          console.log(session?.currentTask)
          return
        }

        const url = decorateUrl('/')
        if (url.startsWith('http')) {
          window.location.href = url
        } else {
          router.push(url as Href)
        }
      },
    })
  } else {
    // Check why the sign-up is not complete
    console.error('Sign-up attempt not complete:', signUp)
  }
}

if (signUp.status === 'complete' || isSignedIn) {
  return null
}

if (
  signUp.status === 'missing_requirements' &&
  signUp.unverifiedFields.includes('email_address') &&
  signUp.missingFields.length === 0
) {
  return (
    <ThemedView style={styles.container}>
      <ThemedText type="title" style={styles.title}>
        Verify your account
      </ThemedText>
      <TextInput
        style={styles.input}
        value={code}
        placeholder="Enter your verification code"
        placeholderTextColor="#666666"
        onChangeText={(code) => setCode(code)}
        keyboardType="numeric"
      />
      {errors.fields.code && (
        <ThemedText style={styles.error}>{errors.fields.code.message}</ThemedText>
      )}
      <Pressable
        style={({ pressed }) => [
          styles.button,
          fetchStatus === 'fetching' && styles.buttonDisabled,
          pressed && styles.buttonPressed,
        ]}
        onPress={handleVerify}
        disabled={fetchStatus === 'fetching'}
      >
        <ThemedText style={styles.buttonText}>Verify</ThemedText>
      </Pressable>
      <Pressable
        style={({ pressed }) => [styles.secondaryButton, pressed && styles.buttonPressed]}
        onPress={() => signUp.verifications.sendEmailCode()}
      >
        <ThemedText style={styles.secondaryButtonText}>I need a new code</ThemedText>
      </Pressable>
    </ThemedView>
  )
}

return (
  <ThemedView style={styles.container}>
    <ThemedText type="title" style={styles.title}>
      Sign up
    </ThemedText>
```

```
      <ThemedText style={styles.label}>Email address</ThemedText>
      <TextInput
        style={styles.input}
        autoCapitalize="none"
        value={emailAddress}
        placeholder="Enter email"
        placeholderTextColor="#666666"
        onChangeText={(emailAddress) => setEmailAddress(emailAddress)}
        keyboardType="email-address"
      />
      {errors.fields.emailAddress && (
        <ThemedText style={styles.error}>{errors.fields.emailAddress.message}</ThemedText>
      )}
      <ThemedText style={styles.label}>Password</ThemedText>
      <TextInput
        style={styles.input}
        value={password}
        placeholder="Enter password"
        placeholderTextColor="#666666"
        secureTextEntry={true}
        onChangeText={(password) => setPassword(password)}
      />
      {errors.fields.password && (
        <ThemedText style={styles.error}>{errors.fields.password.message}</ThemedText>
      )}
      <Pressable
        style={({ pressed }) => [
          styles.button,
          (!emailAddress || !password || fetchStatus === 'fetching') && styles.buttonDisabled,
          pressed && styles.buttonPressed,
        ]}
        onPress={handleSubmit}
        disabled={!emailAddress || !password || fetchStatus === 'fetching'}
      >
        <ThemedText style={styles.buttonText}>Sign up</ThemedText>
      </Pressable>
      {/* For your debugging purposes. You can just console.log errors, but we put them in the UI for cor
      {errors && <ThemedText style={styles.debug}>{JSON.stringify(errors, null, 2)}</ThemedText>}

      <View style={styles.linkContainer}>
        <ThemedText>Already have an account? </ThemedText>
        <Link href="/sign-in">
          <ThemedText type="link">Sign in</ThemedText>
        </Link>
      </View>

      {/* Required for sign-up flows. Clerk's bot sign-up protection is enabled by default */}
      <View nativeID="clerk-captcha" />
    </ThemedView>
  )
}

const styles = StyleSheet.create({
  container: {
    flex: 1,
    padding: 20,
    gap: 12,
  },
  title: {
    marginBottom: 8,
  },
  label: {
    fontWeight: '600',
    fontSize: 14,
  },
  input: {
    borderWidth: 1,
    borderColor: '#ccc',
    borderRadius: 8,
    padding: 12,
    fontSize: 16,
    backgroundColor: '#fff',
  },
  button: {
    backgroundColor: '#0a7ea4',
```

```
    paddingVertical: 12,
    paddingHorizontal: 24,
    borderRadius: 8,
    alignItems: 'center',
    marginTop: 8,
  },
  buttonPressed: {
    opacity: 0.7,
  },
  buttonDisabled: {
    opacity: 0.5,
  },
  buttonText: {
    color: '#fff',
    fontWeight: '600',
  },
  secondaryButton: {
    paddingVertical: 12,
    paddingHorizontal: 24,
    borderRadius: 8,
    alignItems: 'center',
    marginTop: 8,
  },
  secondaryButtonText: {
    color: '#0a7ea4',
    fontWeight: '600',
  },
  linkContainer: {
    flexDirection: 'row',
    gap: 4,
    marginTop: 12,
    alignItems: 'center',
  },
  error: {
    color: '#d32f2f',
    fontSize: 12,
    marginTop: -8,
  },
  debug: {
    fontSize: 10,
    opacity: 0.5,
    marginTop: 8,
  },
})
```

8 Add a `sign-in` page

In the `(auth)` route group, create a `sign-in.tsx` file with the following code. The `useSignIn()` hook is used to create a sign-in flow. The user can sign in using their email and password, or navigate to the sign-up page.

/app/(auth)/sign-in.tsx

```
import { ThemedText } from '@/components/themed-text'
import { ThemedView } from '@/components/themed-view'
import { useSignIn } from '@clerk/expo'
import { type Href, Link, useRouter } from 'expo-router'
import React from 'react'
import { Pressable, StyleSheet, TextInput, View } from 'react-native'

export default function Page() {
  const { signIn, errors, fetchStatus } = useSignIn()
```

9 Add a home screen

You can control which content signed-in and signed-out users can see with Clerk's prebuilt components. For this quickstart, you'll use:

- `<Show when="signed-in">`: Children of this component can only be seen while signed in.

- `<Show when="signed-out">`: Children of this component can only be seen while signed out.

To get started, create a `(home)` route group with a `_layout.tsx` file with the following code.

app/(home)/_layout.tsx

```
import { useAuth } from '@clerk/expo'
import { Redirect, Stack } from 'expo-router'

export default function Layout() {
  const { isSignedIn, isLoaded } = useAuth()

  if (!isLoaded) {
    return null
  }

  if (!isSignedIn) {
    return <Redirect href="/(auth)/sign-in" />
  }

  return <Stack />
}
```

Then, in the same folder, create an `index.tsx` file with the following code. If the user is signed in, it displays their email and the sign-out button you created in the previous step. If they're not signed in, it displays sign-in and sign-up links.

app/(home)/index.tsx

```
import { Show, useUser } from '@clerk/expo'
import { useClerk } from '@clerk/expo'
import { Link } from 'expo-router'
import { Text, View, Pressable, StyleSheet } from 'react-native'

export default function Page() {
  const { user } = useUser()
  const { signOut } = useClerk()
```

Run your project

Run your project with the following command.

npm pnpm yarn bun

terminal

```
npm expo start
```

Then use the terminal shortcuts to run the app on your preferred platform:

- Press i to open the iOS simulator
- Press a to open the Android simulator
- Scan the QR code with Expo Go to run the app on a physical device

Create your first user

Once the app opens on your device or simulator:

- Navigate to the Sign up screen
- Enter your details and complete the authentication flow
- After signing up, your first user will be created and you'll be signed in

Next steps

Protect content and read user data

Learn how to protect content and read session and user data with Expo.

Continue to the Expo guide